This PDF contains 30+ real interview questions and answers covering SQL, .NET, C#, Angular, Azure, and more. All content is based on actual interviews I have researched and compiled to help learners and job aspirants prepare effectively.

I hope this guide is helpful and useful for anyone preparing for interviews. If you find it valuable, please leave a comment or share your feedback. Your support encourages me to create more helpful content!

□ PART 1 – C# & OOP INTERVIEW ANSWERS

1. Why we use using keyword?
It automatically disposes objects (like DB connections, files) that implement IDisposable.
It ensures memory cleanup and prevents resource leaks.

2. Difference between Class and Structure
Class = Reference Type (stored on heap). Struct = Value Type (stored on stack).
Classes support inheritance; structs don't. Structs are used for small data types.

3. Method Overloading vs Overriding
Overloading = Same method name, different parameters (same class).
Overriding = Child class redefines parent method (using virtual and override).

4. Difference between Abstract Class and Interface
Abstract Class = Can have abstract + non-abstract methods.
Interface = Only declarations (no implementation).
Use abstract when you need shared base logic; interface for contract only.

5. Four Pillars of OOP
Encapsulation → Data hiding.
Abstraction → Hide complexity.
Inheritance → Code reuse.
Polymorphism → Different forms of same method.

6. SOLID Principles
S – Single Responsibility, O – Open/Closed, L – Liskov Substitution, I – Interface Segregation, D – Dependency Inversion.
They make code clean, scalable, and maintainable.

7. Difference between Value Type & Reference Type
Value types (store data directly). Reference types (store memory address).
Changing reference affects all references to that object.

8. Equals() vs ==
== checks reference equality (by default).
Equals() can be overridden for value comparison.

9. ref vs out vs in parameters
ref – Pass by reference (must be initialized).

out – Pass by reference (to return multiple values).
in – Pass by reference as read-only.

10. const vs readonly
const – Compile-time constant.
readonly – Runtime constant (set in constructor).

11. What is Boxing/Unboxing
Boxing = Value type → object.
Unboxing = object → value type.
Too much boxing slows performance.

12. What is a Delegate?
A delegate is a type-safe function pointer.
Used for callbacks and event handling.

13. Partial Class
Allows splitting a class into multiple files for organization.
The compiler merges them at build time.

14. Static Class vs Singleton
Static – Cannot instantiate, no state.
Singleton – One instance controlled by class itself using private constructor.

15. Extension Methods
Add new methods to existing types without modifying them.
Use this keyword in the first parameter.

16. Managed vs Unmanaged Code
Managed = Executed by CLR (with GC).
Unmanaged = Outside CLR (e.g., C++, Win32 APIs).

17. What is Serialization?
Converting object → JSON/XML to store or transfer.
Deserialization is the reverse process.

18. Asynchronous Programming (async/await)
Runs tasks without blocking main thread.
Improves performance in I/O operations.

19. Access Specifiers
public, private, protected, internal, protected internal.
They control visibility and access of members.

20. Inheritance Types
Single, Multilevel, Hierarchical, Multiple (via interfaces).
Promotes code reuse and extensibility.

21. What is the difference between .net framework and .net core

The **.NET Framework** is the older, Windows-only version of Microsoft's development platform, while **.NET Core** is the newer, open-source, cross-platform version that supports Windows, Linux, and macOS.

- **.NET Framework** can run only on Windows and is mainly used for enterprise desktop and web applications using IIS.
- **.NET Core** (now known simply as **.NET 5/6/7/8**) can run on any operating system and supports cross-platform deployment.
- .NET Core provides **better performance, scalability, and flexibility**, and it's optimized for **cloud, microservices, and container-based applications**.
- It supports **command-line tools (dotnet CLI)** and **side-by-side versioning**, allowing multiple app versions on the same machine.
- **.NET Framework** is no longer actively developed beyond version 4.8, while **.NET Core / .NET** continues to evolve as the future of the .NET ecosystem.

✅ **Simple summary:**
".NET Framework" is old and limited to Windows.
".NET Core" (now just ".NET") is faster, cross-platform, open-source, and the modern direction of Microsoft development.

22. Custom Middleware in ASP.NET Core

Yes, we can create custom middleware to process requests and responses. It is a class with an Invoke or InvokeAsync method and can perform tasks like logging, authentication, or error handling. Custom middleware allows centralized request/response handling in the pipeline.

23. Web.config and Machine.config

- **Web.config:** Configuration file specific to an ASP.NET web application. Stores settings like connection strings, authentication, and custom app settings.
- **Machine.config:** Global configuration file for the entire .NET Framework on the machine. Provides default settings for all .NET applications.

24. Monolithic Architecture vs Clean Architecture

- **Monolithic Architecture:** Entire application is built as a **single unit**. Easy to develop initially but hard to scale and maintain.
- **Clean Architecture:** Application is divided into **layers (Presentation, Domain, Infrastructure)** with **dependency rules**. Improves testability, maintainability, and scalability.

25. Constructor and Types
A constructor is a **special method** used to **initialize objects** of a class.

Types of Constructors:

- **Default Constructor:** No parameters, initializes default values.
- **Parameterized Constructor:** Accepts parameters to initialize objects with specific values.
- **Static Constructor:** Initializes **static members** of a class.
- **Private Constructor:** Used in **Singleton pattern** to restrict object creation.

26. Serialization and Deserialization

- **Serialization:** Converting an object into a **format that can be stored or transmitted** (like JSON or XML).
- **Deserialization:** Converting **stored or transmitted data back into an object** in memory.

27. Action Filter
An action filter is an **attribute that runs before or after an action method** in ASP.NET Core. It is used for logging, authorization, caching, or modifying the response.

28. Asynchronous Programming in C#
Allows code to **run tasks without blocking the main thread** using async and await, improving performance in I/O-bound operations.

29. Boxing and Unboxing

- **Boxing:** Converting a **value type to an object**.
- **Unboxing:** Converting an **object back to its value type**.

30. ref, out, and in Keywords

- **ref:** Passes a variable by reference; must be initialized before use.
- **out:** Passes a variable by reference; must be assigned inside the method.
- **in:** Passes a variable by reference but **read-only** inside the method.

31. Difference Between Array and ArrayList

- **Array:** Fixed size, strongly typed.
- **ArrayList:** Dynamic size, can store objects of **any type** (less type-safe).

32. Design Patterns
Reusable solutions to common problems in software design. Examples include Singleton, Factory, Repository, Observer.

33. Difference Between PUT and PATCH

- PUT: Updates the entire resource; idempotent (same result on multiple calls).
- PATCH: Updates specific fields of a resource; used for partial updates.

34. Difference Between AddSingleton, AddScoped, and AddTransient

- AddSingleton: Single instance shared across the entire application.
- AddScoped: New instance per HTTP request; shared within the request.
- AddTransient: New instance every time it is requested.

35. Function in C#
A function is a **block of code that performs a specific task** and can return a value. Functions help in **code reusability, modularity, and better organization**.

34. Parallel Programming
Executing **multiple tasks simultaneously** to improve performance and utilize multiple CPU cores efficiently.

35. Difference Between Task and Thread

- **Thread:** Low-level unit of execution managed by OS; heavier.
- **Task:** High-level abstraction over threads; easier to manage and supports **async/await**.

36. Async/Await
Enables **asynchronous programming** in C# to perform non-blocking operations, improving responsiveness in I/O-bound tasks.

37. Difference Between ref and out

- **ref:** Variable must be **initialized before passing**; passed by reference.
- **out:** Variable **does not need initialization**; must be assigned inside the method.

38. What is SOLID Principle?
SOLID is a set of **five design principles** in object-oriented programming to make software **more maintainable, flexible, and scalable**:

- **S – Single Responsibility Principle (SRP):** A class should have **only one reason to change**.
- **O – Open/Closed Principle (OCP):** Software entities should be **open for extension but closed for modification**.

- **L – Liskov Substitution Principle (LSP):** Subtypes should be **replaceable for their base types** without affecting the program.
- **I – Interface Segregation Principle (ISP):** Clients should not be forced to **implement interfaces they don't use**.
- **D – Dependency Inversion Principle (DIP):** Depend on **abstractions, not concrete implementations**.

⚙ PART 2 – ASP.NET CORE & WEB API

1. What is ASP.NET Core?
ASP.NET Core is a cross-platform, open-source framework used to build web apps and APIs with high performance and flexibility.
It's the modern version of ASP.NET, built on .NET 8/7 runtime.

---

2. What is Middleware?
Middleware are software components that handle requests and responses in a pipeline.
Examples: Authentication, Routing, Exception handling, Logging.

---

3. What is Startup.cs used for?
It configures the application pipeline and services.
Two main methods:

- ConfigureServices() → Add services (DI, DB, CORS, etc.)
- Configure() → Define middleware pipeline (UseRouting, UseEndpoints, etc.)

---

4. What is Dependency Injection (DI)?
DI is a design pattern to inject required dependencies instead of creating them manually.
ASP.NET Core has built-in DI container for loose coupling.

---

5. Types of Dependency Injection

- Constructor Injection (most common)
- Method Injection
- Property Injection

---

6. What is the difference between AddSingleton, AddScoped, and AddTransient?

- **Singleton:** One instance for entire app lifetime.
- **Scoped:** One instance per request.
- **Transient:** New instance every time injected.

---

7. What is Routing in ASP.NET Core?
Routing matches incoming URLs to controller actions.
Defined using [Route()], [HttpGet()], etc. or MapControllers().

---

8. What is Attribute Routing?
Routing defined directly using attributes on controllers or actions.
Example: [Route("api/[controller]/[action]")]

---

9. What is Convention-based Routing?
Defined in Startup with patterns like "{controller=Home}/{action=Index}/{id?}".
Less flexible than attribute routing.

---

10. What are Controllers in Web API?
Controllers handle HTTP requests and return responses (usually JSON).
They are classes inherited from ControllerBase.

---

11. What are Action Methods?
Public methods inside controllers that handle specific HTTP verbs.
Example: [HttpGet], [HttpPost], [HttpPut], [HttpDelete].

---

12. What is Model Binding?
Model binding automatically maps incoming request data (JSON, query, form) to method parameters or DTOs.

---

13. What is Model Validation?
It checks if input data is valid using attributes like [Required], [StringLength], [Range].
If invalid, returns 400 Bad Request automatically.

---

14. What are Filters in ASP.NET Core?
Filters allow code to run before or after actions.
Types: Authorization, Action, Result, Exception filters.

---

15. What is Exception Handling Middleware?
Used to catch unhandled exceptions globally.
You can use UseExceptionHandler() or custom middleware to log and return friendly messages.

---

16. What is Entity Framework Core (EF Core)?
ORM tool to work with databases using C# objects instead of SQL queries.
Supports migrations, LINQ, and multiple database providers.

---

17. What is Migration in EF Core?
Migrations manage database schema changes.
Commands:
Add-Migration InitialCreate → creates migration
Update-Database → applies it to DB.

---

18. What is appsettings.json?
Configuration file used to store connection strings, keys, and app settings.
Can be accessed via IConfiguration.

---

19. What is Configuration in ASP.NET Core?
ASP.NET Core uses IConfiguration interface to access values from appsettings.json, environment variables, or secrets.

---

20. What is CORS (Cross-Origin Resource Sharing)?
CORS allows requests from different domains (e.g., Angular app to .NET API).
Enable it using app.UseCors() or [EnableCors].

---

21. What is JWT Authentication?
JWT (JSON Web Token) is a secure token-based authentication method.
Client stores token, sends it in Authorization: Bearer header for each request.

---

22. What are DTOs (Data Transfer Objects)?
DTOs carry data between layers (like API ↔ UI) without exposing entity models.

---

23. What is the difference between IHostedService and BackgroundService?
Both are used for background tasks.
BackgroundService is a base class for long-running background jobs.

---

24. What is app.UseEndpoints()?
Defines the endpoints (controllers, Razor pages, etc.) that handle incoming requests.
Usually placed last in the middleware pipeline.

---

25. What is IWebHostEnvironment?
Gives info about the environment (Development, Staging, Production).
Used to load different configs or error pages based on environment.

---

26. What is Kestrel Server?
Kestrel is the built-in cross-platform web server for ASP.NET Core.
Fast and lightweight; runs behind IIS or standalone.

---

27. What is IActionResult?
Represents the return type of a controller action.
Examples: Ok(), NotFound(), BadRequest(), Created().

---

28. How to return custom status codes in Web API?
Use built-in methods like:
return Ok(data);
return NotFound("Not found");
return StatusCode(500, "Server error");

---

29. What is Swagger in ASP.NET Core?
Swagger (via Swashbuckle) auto-generates API documentation and allows testing endpoints through a web UI.

□ PART 3 – SQL & ENTITY FRAMEWORK CORE

◈ SQL INTERVIEW QUESTIONS

1. What is SQL?
SQL (Structured Query Language) is used to store, manage, and retrieve data from databases.

2. What are the main types of SQL statements?

- DDL (Data Definition Language): CREATE, ALTER, DROP
- DML (Data Manipulation Language): INSERT, UPDATE, DELETE
- DCL (Data Control Language): GRANT, REVOKE
- TCL (Transaction Control Language): COMMIT, ROLLBACK

3. What is a Primary Key?
A unique identifier for each record in a table. Cannot be null or duplicated.

4. What is a Foreign Key?
A column that links data between two tables (parent-child relationship) and enforces referential integrity.

5. What is a Unique Key?
Ensures all values in a column are unique; allows one null value.

6. What is a Join in SQL?
Used to combine rows from two or more tables based on a related column.

7. What are the types of Joins in SQL?

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN
- SELF JOIN

8. Difference Between INNER JOIN and LEFT JOIN

- INNER JOIN: Returns only matching rows.
- LEFT JOIN: Returns all rows from the left table + matched rows from right table.

9. Difference Between LEFT JOIN and RIGHT JOIN

- LEFT JOIN: Returns all rows from the left table.
- RIGHT JOIN: Returns all rows from the right table.

10. What is a FULL OUTER JOIN?
Returns all rows from both tables, with NULLs where there is no match.

11. What is a CROSS JOIN?
Returns the Cartesian product of two tables (every row from first table combined with every row from second table).

12. What is a SELF JOIN?
A join where a table is joined with itself, useful for hierarchical data.

13. What is a Subquery?
A query inside another query, used to filter or calculate data.

14. Types of Subqueries

- Single-row subquery
- Multi-row subquery
- Correlated subquery

15. Difference Between Candidate Key, Primary Key, and Super Key

- Candidate Key: Unique key that can be primary.
- Primary Key: Selected candidate key to uniquely identify a row.
- Super Key: Set of columns that uniquely identify a row.

16. Difference Between WHERE and HAVING

- WHERE: Filters rows before aggregation.
- HAVING: Filters rows after aggregation (used with GROUP BY).

17. Difference Between DELETE, TRUNCATE, and DROP

- DELETE: Removes rows; can use WHERE; can be rolled back.
- TRUNCATE: Removes all rows; faster; minimal logging; cannot use WHERE; cannot always be rolled back.
- DROP: Deletes the entire table structure and data permanently.

18. What are ACID Properties in SQL?

- A – Atomicity: All or nothing transaction.
- C – Consistency: Database remains valid before and after transaction.
- I – Isolation: Transactions do not interfere.
- D – Durability: Committed transactions are permanent.

19. What is a Clustered Index?
Data is physically stored in the order of the index (one per table).

20. What is a Non-Clustered Index?
Separate structure that points to the data; can have multiple per table.

21. What is a View in SQL?
A virtual table based on the result of a query; simplifies complex queries and improves security.

22. What is a Stored Procedure?
Precompiled SQL code stored in the database; improves performance and reusability.

23. What is a Function in SQL?
A reusable SQL code that returns a single value or table.

24. Difference Between Stored Procedure and Function

- Stored Procedure: Can perform actions like INSERT, UPDATE, DELETE; may or may not return value.
- Function: Must return a value; cannot perform DML operations directly.

25. What is a Cursor in SQL?
A database object used to fetch rows one at a time from a result set.

26. What is a Trigger in SQL?
A piece of SQL code executed automatically after INSERT, UPDATE, or DELETE.

27. What are Magic Tables in SQL?
Temporary tables used in triggers to store old and new values (INSERTED and DELETED).

28. What is Normalization?
Organizing data to reduce redundancy and improve integrity (1NF → 2NF → 3NF).

29. What are Normal Forms (1NF, 2NF, 3NF, BCNF)?

- 1NF: Eliminate duplicate columns; ensure atomicity.
- **2NF: Eliminate partial dependency.**
- **3NF: Eliminate transitive dependency.**
- **BCNF: Every determinant is a candidate key.**

30. What is Denormalization?
Adding redundancy for faster read performance, often used in reporting.

31. What is a Composite Key?
A key made up of two or more columns to uniquely identify a row.

32. What is a Candidate Key?
A column or set of columns that can uniquely identify a row.

33. What is a Super Key?
A set of columns that uniquely identifies a row in a table.

34. What is a Surrogate Key?
An artificial key, usually auto-increment, used to uniquely identify a row.

35. What is Data Integrity in SQL?
Ensures accuracy and consistency of data through constraints and rules.

36. Difference Between UNION and UNION ALL

- **UNION: Removes duplicates.**
- **UNION ALL: Includes duplicates.**

37. Difference Between DISTINCT and GROUP BY

- **DISTINCT: Removes duplicate rows.**
- **GROUP BY: Groups rows for aggregation.**

38. Aggregate Functions in SQL (COUNT, SUM, AVG, MIN, MAX)
Functions that perform calculations on a set of values.

39. Difference Between CHAR and VARCHAR

- **CHAR: Fixed length.**
- **VARCHAR: Variable length.**

40. How to Find the Nth Highest Salary
Use subquery or ROW_NUMBER():
SELECT DISTINCT Salary FROM Employees e1 WHERE N = (SELECT COUNT(DISTINCT Salary) FROM Employees e2 WHERE e2.Salary >= e1.Salary)

41. Query to Get Highest Salary from Each Department
SELECT DepartmentId, MAX(Salary) AS HighestSalary FROM Employees GROUP BY DepartmentId;

42. Query to Select Duplicate Rows
SELECT Name, COUNT(*) FROM Employees GROUP BY Name HAVING COUNT(*) > 1;

43. Query to Select the Second Highest Salary
SELECT MAX(Salary) FROM Employees WHERE Salary < (SELECT MAX(Salary) FROM Employees);

44. Query to Calculate Total Marks of All Subjects per Student
SELECT StudentId, SUM(Marks) AS TotalMarks FROM MarksTable GROUP BY StudentId;

45. Indexing and Its Importance
Indexes improve query performance by allowing faster data retrieval; common types: Clustered, Non-Clustered, Unique.

46. Difference between SQL and NoSQL Databases

- **SQL (Relational):** Uses structured tables with fixed schemas; supports ACID transactions; ideal for complex queries and structured data.

- **NoSQL (Non-Relational):** Uses document, key-value, column, or graph models; schema-less; designed for scalability and handling unstructured or semi-structured data.
- **Use Case:** SQL for traditional applications (banking, ERP); NoSQL for big data, real-time apps, and flexible schema requirements.

47. Temporary Table in SQL

- Definition: A temporary table is a table that is created and used to store intermediate results temporarily during a session or a procedure.
- Scope: Exists only for the duration of the session or procedure; automatically dropped when the session ends (local) or when all sessions are done (global).
- Use Case: Used to simplify complex queries, store intermediate results, or perform calculations without affecting permanent tables.

48. Difference between Function and Stored Procedure

- **Function**: Returns a single value or table and can be used in SQL queries like SELECT. Cannot perform actions like INSERT, UPDATE, or DELETE on tables in most cases.
- **Stored Procedure**: Performs a set of operations (can include INSERT, UPDATE, DELETE) and may or may not return values. Cannot be used directly inside a SELECT statement.

Key Points:

- Functions are primarily for computation and returning values.
- Stored Procedures are for executing a sequence of SQL statements.
- Functions can be called from other SQL statements; stored procedures are executed using EXEC or EXECUTE.

◈ ENTITY FRAMEWORK CORE (EF CORE) INTERVIEW QUESTIONS

21. What is Entity Framework Core?
It's Microsoft's ORM for .NET to work with databases using C# objects instead of SQL.

---

22. What are DbContext and DbSet?

- **DbContext:** Bridge between database and C# objects.
- **DbSet:** Represents a table in the database.

---

23. What are the advantages of EF Core?

- Eliminates SQL writing
- Supports migrations
- Cross-database (SQL Server, SQLite, MySQL)
- Easy to maintain

---

24. What is Code First approach?
You create C# classes first, and EF Core generates the database using migrations.

---

25. What is Database First approach?
You start with an existing database, and EF Core generates models automatically.

---

26. What are Migrations in EF Core?
They track and apply schema changes from code to database.
Commands:
Add-Migration <name> → create migration
Update-Database → apply it

---

27. How to define relationships in EF Core?

- **One-to-One** → HasOne().WithOne()
- **One-to-Many** → HasMany().WithOne()
- **Many-to-Many** → HasMany().WithMany()

---

28. What are Navigation Properties?
They allow navigation between related entities using foreign keys (like Student → Address list).

---

29. What is Lazy Loading vs Eager Loading?

- **Lazy Loading:** Loads related data only when accessed.
- **Eager Loading:** Loads related data immediately using .Include().

---

30. What is the difference between SaveChanges() and SaveChangesAsync()?
SaveChanges() is synchronous.
SaveChangesAsync() is non-blocking and improves scalability.

---

31. What are Shadow Properties?
Properties not defined in entity class but tracked by EF Core (e.g., CreatedDate).

---

32. What is Change Tracking?
EF Core tracks changes made to entities and saves only modified data during SaveChanges().

---

33. What is LINQ?
(Language Integrated Query) – allows querying collections or databases in C# syntax.
Example:

```
var students = context.Students.Where(x => x.Age > 18).ToList();
```

---

34. What is Difference Between ToList(), FirstOrDefault(), and SingleOrDefault()?

- ToList() → returns all matching items.
- FirstOrDefault() → first match or null.
- SingleOrDefault() → expects exactly one match or null (throws error if multiple).

---

35. How to include related data in EF Core?
Use .Include() and .ThenInclude()
Example:

```
context.Students.Include(s => s.Addresses).ToList();
```

---

36. How to delete related data (cascade delete)?
Configure in model:

```
modelBuilder.Entity<Student>()

.HasMany(s => s.Addresses)
```

```
.WithOne(a => a.Student)

.OnDelete(DeleteBehavior.Cascade);
```

---

37. How to handle exceptions in EF Core?
Use try-catch around SaveChanges() and log DbUpdateException or SqlException.

---

38. What is Raw SQL Query in EF Core?
You can execute SQL manually:

```
context.Students.FromSqlRaw("SELECT * FROM Students");
```

---

39. What is DbContext Lifecycle?
DbContext should be short-lived — per request or per operation.
Never use a single context globally.

---

40. How to Seed Initial Data in EF Core?
Use HasData() inside OnModelCreating():

```
modelBuilder.Entity<Student>().HasData(new Student { Id=1, Name="Nifty" });
```

41. DbContext in EF Core
DbContext is the **primary class** for interacting with the database in EF Core. It **manages entity objects, tracks changes, and handles CRUD operations**.

42. DTO (Data Transfer Object) and Its Purpose
A DTO is an **object that carries data between processes**. It is used to **transfer only required data**, reduce payload, and improve security by hiding internal models.

43. EF Core Migration
Migrations in EF Core are used to **incrementally update the database schema** to match your data model. They help in **creating, updating, and version-controlling database changes**.

44. Disadvantages of Entity Framework

- **Performance overhead:** Slower than raw SQL for complex queries.
- **Learning curve:** Requires understanding of ORM concepts and EF-specific features.
- **Less control over SQL:** Automatically generated queries may not be fully optimized.
- **Complex debugging:** Harder to trace issues due to abstraction layers.
- **Memory usage:** Can consume more memory for tracking entities in large applications.

💻 PART 4 – ANGULAR INTERVIEW QUESTIONS & ANSWERS

🔷 Basics

1. What is Angular?
Angular is a TypeScript-based framework for building single-page web applications (SPAs).
It's maintained by Google and supports two-way data binding, dependency injection, and modular structure.

---

2. What is TypeScript and why used in Angular?
TypeScript is a superset of JavaScript with static typing.
It helps catch errors early and makes large-scale apps easier to maintain.

---

3. What is a Component in Angular?
A component is the basic building block of Angular apps — it controls a part of the UI.
It includes HTML (template), CSS (style), and TypeScript (logic).

---

4. What are the main building blocks of Angular?
1️⃣ Modules
2️⃣ Components
3️⃣ Templates
4️⃣ Directives
5️⃣ Services
6️⃣ Dependency Injection

---

5. What is a Module?
A module (@NgModule) is a container that groups related components, directives, pipes, and services.
Main module: AppModule.

---

6. What is Data Binding in Angular?
Connecting data between component and template.
Types:

- **Interpolation:** {{value}}

- **Property Binding:** [property]="value"
- **Event Binding:** (event)="method()"
- **Two-way Binding:** [(ngModel)]="value"

---

7. What is Two-Way Data Binding?
It keeps the UI and component synchronized automatically using [(ngModel)].
Example:

```
<input [(ngModel)]="studentName">
```

---

8. What is a Directive?
Directives are instructions in the DOM.

- **Structural:** *ngIf, *ngFor
- **Attribute:** [ngClass], [ngStyle]

---

9. What is a Service in Angular?
A service holds reusable business logic or data fetching code.
It's injected into components via dependency injection.

---

10. What is Dependency Injection in Angular?
DI automatically provides required services to components.
Helps in loose coupling and testability.

---

11. What is the purpose of @Injectable() decorator?
It marks a class as available for dependency injection.

---

12. What is Routing in Angular?
Routing allows navigation between views or pages without reloading.
Defined in app-routing.module.ts.

---

13. What is a Router Outlet?
A placeholder in HTML that loads the routed component's template.

```
<router-outlet></router-outlet>
```

---

14. What are RouterLink and RouterLinkActive?
[routerLink] – used to navigate between routes.
routerLinkActive – applies CSS class when route is active.

---

15. What is Lazy Loading?
It loads feature modules only when needed — improving performance.
Configured in routing using loadChildren.

---

16. What is an Observable?
An Observable is a stream of asynchronous data (like API responses).
Handled using RxJS.

---

17. What is RxJS?
Reactive Extensions for JavaScript – provides operators like map, filter, subscribe, etc., for managing async data streams.

---

18. What is HttpClient in Angular?
It's used to make HTTP requests (GET, POST, PUT, DELETE).
Example:

```
this.http.get('api/students').subscribe();
```

---

19. What are Promises vs Observables?

- **Promise:** Handles one value (resolved once).
- **Observable:** Handles multiple values (streams, can cancel).

---

20. What is Interceptor in Angular?
An interceptor modifies HTTP requests/responses (e.g., adding JWT tokens).
Implemented using HttpInterceptor.

---

21. What is a Pipe in Angular?
Pipes transform data in templates.
Example: {{ name | uppercase }}
Custom pipes can be created with @Pipe() decorator.

---

22. What are Lifecycle Hooks?
Methods that execute at specific times in a component's life.
Common ones:

- ngOnInit() – runs once after component loads.
- ngOnDestroy() – runs before destruction.
- ngOnChanges() – runs when input changes.

---

23. What is ngOnInit() used for?
Used for initialization logic (like API calls) after component creation.

---

24. What is Event Binding?
Used to handle user actions like clicks.
Example:

```
<button (click)="save()">Save</button>
```

---

25. Difference between Template-driven and Reactive Forms?

- **Template-driven:** Simple forms using [(ngModel)]
- **Reactive forms:** More powerful, use FormGroup, FormControl, and FormBuilder.

---

26. What is FormGroup and FormControl?
FormGroup represents the entire form, FormControl represents individual input.
Example:

```
form = new FormGroup({

 name: new FormControl(''),

 age: new FormControl('')

});
```

---

27. What is FormBuilder?
A service to build forms easily.
Example:

```
this.form = this.fb.group({

 name: [''],
```

```
 age: ['']
});
```

---

28. What is ngFor and ngIf?

- *ngFor – loops through arrays.
- *ngIf – conditionally displays elements.

---

29. What is ViewChild?
Used to access child component or DOM element from parent class.
Example:

```
@ViewChild('formRef') form: NgForm;
```

---

30. What is a Guard in Angular?
Guards protect routes.
Types: CanActivate, CanDeactivate, Resolve, CanLoad.
Used for authentication and permission checks.

---

31. What is Change Detection?
It updates the UI automatically when data changes.
Angular does this using its Zone.js mechanism.

---

32. What is the purpose of ngZone?
NgZone lets you run code inside or outside Angular's change detection mechanism to improve performance.

---

33. What is the difference between AngularJS and Angular?

- AngularJS → JavaScript-based, MVC pattern.
- Angular → TypeScript-based, component architecture, faster.

---

34. What is a Single Page Application (SPA)?
An SPA loads one main HTML page and dynamically updates content without reloading the page.

---

35. What are Modules types in Angular?

- **Root Module (AppModule)**
- **Feature Module**
- **Shared Module**
- **Core Module**

---

36. How to share data between components?
1. Input & Output decorators
2. Services (shared state)
3. LocalStorage
4. Route parameters

37. What is AOT Compilation?
Ahead-of-Time compilation compiles code at build time, improving performance and reducing load time.

---

38. How to call a .NET API from Angular?
Use HttpClient:

```
this.http.get('https://localhost:5001/api/Students').subscribe(data => console.log(data));
```

39. What is Angular and Difference from AngularJS

Angular is a **TypeScript-based framework** for building single-page applications (SPA). Unlike AngularJS, it uses **components, TypeScript, and improved performance**.

40. Architecture of an Angular Application

Angular architecture is **component-based** and includes **Modules, Components, Templates, Services, and Dependency Injection**.

41. One-Way Binding vs Two-Way Binding

- **One-way binding:** Data flows **from component to template** only.
- **Two-way binding:** Data flows **both ways**, keeping component and template in sync.

42. Purpose of Angular Services

Services are used to **share data and logic across components**, such as API calls or reusable functions.

43. Angular Component Lifecycle

Angular components go through **lifecycle hooks** like ngOnInit, ngOnChanges, ngOnDestroy to manage initialization, changes, and cleanup.

44. Handling Forms in Angular

Angular handles forms using **Template-driven or Reactive Forms** for validation and data binding.

45. Dependency Injection in Angular

Angular DI allows **injecting services into components** for **loose coupling and reusability**.

46. Async Pipe in Angular

Async pipe **automatically subscribes to observables or promises** and updates the template when data changes.

47. Handling API Calls in Angular

API calls are handled using **HttpClient** with observables to fetch, post, update, or delete data.

48. Difference between @ViewChild and @ViewChildren

- **@ViewChild:** Accesses a **single child element/component**.
- **@ViewChildren:** Accesses **multiple child elements/components** as a QueryList.

49. Handling Errors in Angular

Errors are handled using **catchError in RxJS** or **global error interceptors**.

50. AOT (Ahead-of-Time) Compilation in Angular

AOT compiles **TypeScript and templates to JavaScript** during build time, improving performance.

51. Angular Guards

Guards **control access to routes** based on conditions like authentication or roles.

52. Observable vs Promise

- **Observable:** Can emit **multiple values over time** and supports operators.
- **Promise:** Emits **a single value** and cannot be canceled.

53. Role of NgZone Service

NgZone helps Angular **detect and trigger change detection** when asynchronous operations occur outside Angular's context.

54. Lazy Loading in Angular

Lazy loading **loads modules only when needed**, improving app performance by reducing initial load time.

55. Angular Interceptors

Interceptors allow **modifying HTTP requests or responses globally**, useful for adding tokens, logging, or error handling.

56. Ensuring State Management in Angular

State can be managed using **NgRx, BehaviorSubject, or services** to maintain consistent data across components.

57. Sharing Data Between Components

Data is shared using **services, Input/Output decorators, or state management libraries**.

58. Angular Universal

Angular Universal enables **server-side rendering (SSR)** for better SEO and faster initial load.

59. Difference Between switchMap, mergeMap, concatMap, and exhaustMap

- **switchMap:** Cancels previous observable on new emission.
- **mergeMap:** Runs multiple observables **concurrently**.
- **concatMap:** Runs observables **sequentially**, one after another.
- **exhaustMap:** Ignores new observables until the current one completes.

60. Difference Between Resolver and Guard in Angular Routing

- **Resolver:** Fetches data **before activating a route**.
- **Guard:** Controls **access to a route** based on conditions like authentication.

61. What is a Form in Angular and its Types?

A **Form** in Angular is used to capture and manage user input. Angular provides two main types of forms:

- **Template-driven Forms**: Defined in the HTML template using directives like ngModel. Simpler and suitable for basic forms.
- **Reactive Forms**: Defined in TypeScript using FormGroup, FormControl, and FormArray. Provides more control, validation, and scalability for complex forms.

---

63. How do we Pass Token in Angular (Interceptor)?

- Use an **HTTP Interceptor** to attach tokens (like JWT) to every HTTP request automatically.
- Steps:
    1. Create a class implementing HttpInterceptor.
    2. Use intercept() method to clone the request and add the Authorization header with the token.
    3. Provide the interceptor in app.module.ts under HTTP_INTERCEPTORS.

Purpose: Ensures secure API calls without manually adding tokens in each request.

☁ PART 5 – AZURE, DEVOPS, JWT & HR QUESTIONS

◈ AZURE & CLOUD BASICS

1. What is Microsoft Azure?
Azure is Microsoft's cloud platform that provides services like computing, storage, databases, networking, and AI on demand.

---

2. What are the main Azure service types?

- **IaaS (Infrastructure as a Service)** – Virtual Machines
- **PaaS (Platform as a Service)** – App Service, Azure SQL
- **SaaS (Software as a Service)** – Office 365, Outlook

---

3. What is Azure App Service?
It's a fully managed PaaS service to host web apps, REST APIs, and mobile backends.

---

4. What is Azure Function?
Azure Function is a serverless compute service that runs small pieces of code in response to events — you pay only when it runs.

---

5. What is Azure Blob Storage?
Blob storage is used to store large amounts of unstructured data (like images, videos, backups, etc.).

---

6. What is Azure SQL Database?
It's a fully managed relational database as a service (PaaS) — supports automatic backup, scaling, and high availability.

---

7. What is Azure Virtual Machine?
VM is an IaaS service that provides scalable computing power in the cloud — like having your own server on Azure.

---

8. What is Azure DevOps?
Azure DevOps is a set of tools for CI/CD pipelines, version control, and project management (Boards, Repos, Pipelines, Test Plans, Artifacts).

---

9. What is CI/CD Pipeline?

- **CI (Continuous Integration):** Build and test code automatically on every push.
- **CD (Continuous Deployment):** Automatically deploy tested code to staging or production.

---

10. What are the main components of Azure DevOps?
1⃣ **Repos** – Source control
2⃣ **Pipelines** – Build & deploy
3⃣ **Boards** – Project tracking
4⃣ **Artifacts** – Package management
5⃣ **Test Plans** – Manual & automated testing

11. Main Components of Azure DevOps

1. **Repos:** Source control for code.
2. **Pipelines:** Build, test, and deploy applications.
3. **Boards:** Project and work item tracking.
4. **Artifacts:** Package management and sharing.
5. **Test Plans:** Manual and automated testing.

12. Azure Function and Its Use
Azure Function is a **serverless compute service** to run code on-demand. Use it for **event-driven tasks, background jobs, or lightweight APIs** without managing infrastructure.

13. Azure Service Bus
A **messaging service** for connecting applications and services. Supports **queues and topics** to ensure reliable communication and decoupling between systems.

14. How CI/CD Works in Azure DevOps

- **CI (Continuous Integration):** Automatically builds and tests code on commits.
- **CD (Continuous Deployment/Delivery):** Automatically deploys code to environments.
- Ensures **faster, reliable, and repeatable releases**.

🔷 SECURITY & JWT

11. What is JWT (JSON Web Token)?
JWT is a compact, secure token used for authentication between client and server.
It contains user info in a digitally signed form (Header, Payload, Signature).

---

12. What are the parts of a JWT token?
1⃣ **Header:** Type & algorithm (e.g., HS256)
2⃣ **Payload:** User data or claims
3⃣ **Signature:** Verifies token integrity

---

13. How JWT works in .NET API?

- User logs in → Server generates token → Token is sent to client
- Client stores token → Sends it in Authorization: Bearer <token>
- Server validates it on each request

---

14. Where should JWT be stored on the client side?
In **localStorage** or **sessionStorage** (never in cookies for APIs).

---

15. How to secure Web APIs in ASP.NET Core?

- Use JWT Authentication
- Validate input models
- Use HTTPS
- Implement role-based authorization
- Avoid exposing sensitive info

16. REST API Architecture Style:
REST is a stateless client-server architecture that uses standard HTTP methods (GET, POST, PUT, DELETE) to interact with resources. It is lightweight, scalable, and exchanges data usually in JSON format.

17. HTTP Methods and Their Purpose

- **GET:** Retrieves data from the server without changing it.
- **POST:** Sends data to the server to create a new resource.
- **PUT:** Updates an existing resource completely on the server.
- **PATCH:** Updates part of an existing resource on the server.

- **DELETE:** Removes a resource from the server.
- **OPTIONS:** Returns supported HTTP methods for a resource.
- **HEAD:** Retrieves only the headers of a resource, not the body.

✅ **In short:** Each HTTP method defines how the client communicates with the server and what action is performed on the resource.

18. Middleware in ASP.NET Core Web API

Middleware is a component in the **request-response pipeline** that can **process, modify, or handle HTTP requests and responses**. It's used for tasks like **authentication, logging, error handling, and routing**.

✅ **In short:** Middleware sits between the client and server and controls how requests are handled and responses are sent.

19. How to Secure ASP.NET Core Web API

We secure ASP.NET Core Web API using **Authentication and Authorization**. Common methods include **JWT tokens, OAuth2, API keys**, and implementing **role-based or policy-based access control**.

✅ **In short:** Authentication verifies the user, and authorization controls what they can access.

20. Model Binding in ASP.NET Core Web API

Model Binding is the process where **ASP.NET Core automatically maps HTTP request data** (like query strings, form data, route values, or JSON body) to **action method parameters or model objects**.

✅ **In short:** It makes it easy to receive and use client data in API methods without manual parsing.

21. Dependency Injection (DI) in ASP.NET Core

Dependency Injection is a design pattern where **dependencies are provided to a class instead of the class creating them itself**.

Benefits:

- Promotes **loose coupling**
- Makes code **testable and maintainable**
- Improves **reusability and flexibility**

✅ **In short:** DI allows easier management of dependencies and cleaner, modular code.

22. Error Handling in ASP.NET Core Web APIs

Errors in ASP.NET Core Web APIs are handled using **Middleware** like UseExceptionHandler or UseDeveloperExceptionPage, and **try-catch blocks** in controllers. You can also implement **global exception handling** with custom middleware to return consistent error responses.

✅ **In short:** Use middleware and structured exception handling to catch errors and provide meaningful responses.

23. Data Annotations in ASP.NET Core Web API

Data Annotations are **attributes applied to model properties** to enforce **validation rules, formatting, and metadata**. Common examples include [Required], [MaxLength], [Range], and [EmailAddress].

✅ **In short:** They help validate incoming data automatically and make models self-descriptive.

26. Database Migration in EF Core

In EF Core, database migrations are performed using the **CLI commands**:

1. Add-Migration <MigrationName> – creates a new migration script based on model changes.
2. Update-Database – applies the migration to the database.

✅ **In short:** Migrations let you **safely update the database schema** as your models evolve.

27. Rollback Migrations in EF Core

You can rollback a migration using the **Update-Database <PreviousMigrationName>** command in the Package Manager Console, or use **dotnet ef database update <PreviousMigrationName>** with the CLI.

✅ **In short:** This restores the database schema to a **previous state** before the unwanted migration.

28. Action Filter in ASP.NET Core

An Action Filter is a **custom attribute** that lets you run code **before or after an action method executes** in a controller. It's commonly used for **logging, validation, authentication, or modifying responses**.

✅ **In short:** Action Filters help you **inject reusable logic** around controller actions without changing the action code.

29. Logging in ASP.NET Core Web API

Logging in ASP.NET Core is done using the built-in **ILogger** interface. You can log **information, warnings, errors, or critical messages** by injecting ILogger into controllers or services and calling methods like LogInformation(), LogWarning(), and LogError().

✅ **In short:** ILogger provides a **structured way to record runtime information** for debugging and monitoring.

30. CORS in ASP.NET Core

CORS (Cross-Origin Resource Sharing) is a security feature that **allows or restricts web applications running on one domain from accessing resources on another domain**.

How to enable: Use the **AddCors** method in Startup.cs to configure policies and **UseCors** in the request pipeline. Example:

```
services.AddCors(options => {

   options.AddPolicy("AllowAll", builder =>
builder.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());

});

app.UseCors("AllowAll");
```

✅ **In short:** CORS controls **which domains can access your API** and prevents unauthorized cross-origin requests.

31. API Versioning in ASP.NET Core Web API

API Versioning allows you to **maintain multiple versions of your API** simultaneously. You can implement it using the **Microsoft.AspNetCore.Mvc.Versioning** package and configure versioning via **URL, query string, or HTTP header**.

✅ **In short:** API Versioning helps you **update APIs without breaking existing clients**.

32. Using appsettings.json in ASP.NET Core

appsettings.json is used to **store configuration data** like connection strings, API keys, and custom settings. You can access it via **IConfiguration** in your application.

✅ **In short:** It provides a **centralized way to manage application settings**.

33. What is Swagger and why do we use it in ASP.NET Core?

Swagger is an **open-source tool** used to **generate, describe, and test RESTful APIs**.
In ASP.NET Core, it's commonly used through the **Swashbuckle** library to automatically create **interactive API documentation**.

It helps developers and testers **understand, test, and consume APIs** without writing a single line of extra documentation.

34. HTTPClient Factory

HttpClientFactory is used to **create and manage HttpClient instances** in .NET Core efficiently. It **prevents socket exhaustion** and allows configuration, logging, and resiliency policies.

35. System Versioning in SQL

System-versioned tables in SQL **automatically track historical data changes** with start and end time columns, enabling temporal queries.

36. One-to-One Relationship in EF Core

A one-to-one relationship connects **two entities where each entity has exactly one related entity**.

37. One-to-Many Relationship in EF Core

A one-to-many relationship connects **one entity to multiple related entities**.

38. Many-to-Many Relationship in EF Core

A many-to-many relationship connects **multiple entities on both sides**, usually via a join table.

39. How to Set Unique Key

A unique key is used to **enforce uniqueness on a column or combination of columns** in a database table.

40. Role of HTTPS in Security

HTTPS (Hypertext Transfer Protocol Secure) **encrypts data** between the client and server using SSL/TLS. It **prevents data interception, tampering, and ensures secure communication** over the internet.

✅ **In short:** HTTPS protects sensitive information and maintains data integrity and confidentiality.

41. File Upload in ASP.NET Core Web API

File upload is handled using **IFormFile** in controller actions, which allows receiving files from HTTP requests.

42. Authentication using Identity in ASP.NET Core Web API

ASP.NET Core Identity provides **user registration, login, and role-based authentication** to secure APIs.

43. Global Exception Handling

Global exception handling is implemented using **middleware** to catch all unhandled exceptions and return consistent error responses.

44. Difference between IActionResult and ActionResult

- IActionResult is an **interface** representing the result of an action.
- ActionResult is a **concrete class** implementing IActionResult with helper methods like Ok(), NotFound(), etc.

45. Optimizing API Performance

API performance is improved using **caching, AsNoTracking for EF queries, pagination, and compiled queries**.

46. Attribute Routing in ASP.NET Core Web API

Attribute routing allows defining **routes directly on controllers or actions** using attributes like [Route] or [HttpGet("path")].

47. Limitations/Disadvantages of JWT Token

- **Cannot be easily revoked:** Once issued, JWT tokens are valid until expiration.
- **Token size is large:** Contains all user info and claims, increasing request payload.
- **Security risk if compromised:** If the secret key is exposed, attackers can forge tokens.
- **No built-in encryption:** Data inside JWT is only base64-encoded, not encrypted.
- **Cannot store sensitive info:** Sensitive data should not be stored in JWT payload.

48. Benefits of RESTful Services

- Stateless communication simplifies server design and scalability.
- Lightweight and uses standard HTTP methods for CRUD operations.
- Supports multiple formats like JSON and XML, making it flexible.
- Easy to integrate with web, mobile, and cloud applications.

---

49. Principles of RESTful APIs

- **Stateless:** Each request contains all required information.
- **Client-Server Architecture:** Separation of client and server concerns.
- **Uniform Interface:** Standard methods (GET, POST, PUT, DELETE).
- **Cacheable:** Responses can be cached to improve performance.
- **Layered System:** Intermediary servers can be used without affecting requests.
- **Code on Demand (optional):** Servers can provide executable code to clients.

---

50. Token Expiry and Its Usage in Security

- **Definition:** Token expiry sets a time limit for how long a token is valid.
- **Purpose:** Protects against unauthorized access if tokens are stolen.
- **Usage:** After expiry, users must re-authenticate to obtain a new token.

---

51. Difference between Authentication and Authorization

- **Authentication:** Verifies the identity of a user (Who you are).
- **Authorization:** Determines what resources or actions a user can access (What you can do).

---

52. How Caching is Implemented in .NET Core

- **In-Memory Caching:** Stores data in server memory for quick access.
- **Distributed Caching:** Stores cache across multiple servers for scalability (e.g., Redis).
- **Response Caching:** Caches HTTP responses to reduce repeated processing.